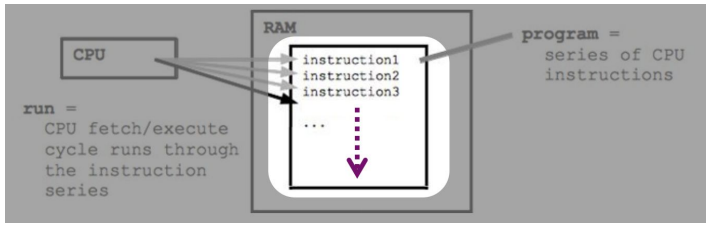
CSCI 12532
Computer Architecture and Design
Lecture 05

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

1. Machine Instructions

- A computer is governed by instructions.
- To perform a given task, a program consisting of a list of machine instructions is stored in the memory. Data to be used as operands are also stored in the memory.
- Individual instructions are brought from the memory into the processor, one after another, in a sequential way (normally).
- The processor executes the specified operation/instruction.



The tasks carried out by a computer program consist of a sequence of machine instructions.

Machine instructions must be capable of performing the following four types of operations:

1) Data transfer between memory and processor registers

2) Arithmetic and logical operations on data in processor

3) Program sequencing and control (e.g. branches, subroutine calls)

4) I/O transfers

Machine Instructions notations

1. Register Transfer Notation (RTN)

Describes the data transfer from one location in computer to another.

– Possible locations: memory locations, processor registers. Locations can be identified symbolically with names (e.g. LOC).

Ex: R2 ←[LOC]

2. Assembly-Language Notation

Is used to represent machine instructions and programs. An instruction must specify an operation to be performed and the operands involved.

Sometimes operations are defined by using mnemonics. Mnemonics: abbreviations of the words describing operations

Ex. The instruction that causes the transfer from memory location LOC to register R2:

Load R2, LOC

Load: *operation*;

LOC: *source operand;*

R2: *destination operand.*

Activity

Given an ***Add*** instruction that

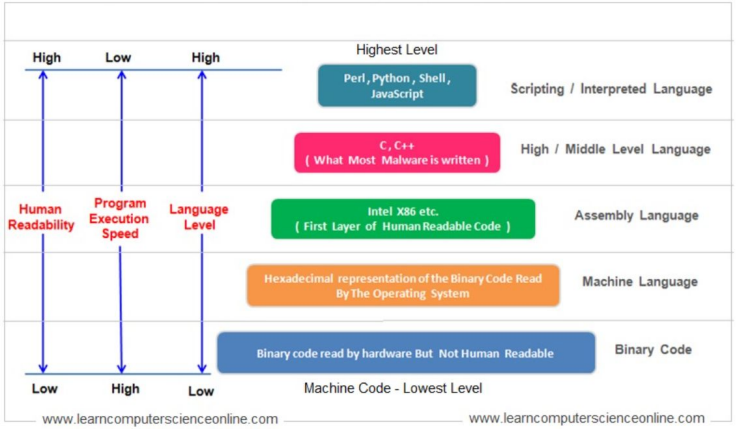
1. Adds the contents of registers R2 and R3, and
2. Places the sum into R4.

Represent this instruction by using

– Register Transfer Notation (RTN)

– Assembly-Language Notation

Types of Programming Languages



Elements of an Instruction

Each instruction must contain the information required by the processor for execution.

■ Operation code: Specifies the operation to be performed (e.g., ADD, I/O). The operation is specified by a binary code, known as the operation code, or opcode.

■ Source operand reference: The operation may involve one or more source operands, that is, operands that are inputs for the operation.

■ Result operand reference: The operation may produce a result.

■ Next instruction reference: This tells the processor where to fetch the next instruction after the execution of this instruction is complete.

The address of the next instruction to be fetched could be either a real address or a virtual address, depending on the architecture. Generally, the distinction is transparent to the instruction set architecture. In most cases, the next instruction to be fetched immediately follows the current instruction. In those cases, there is no explicit reference to the next instruction. When an explicit reference is needed, the main memory or virtual memory address must be supplied.

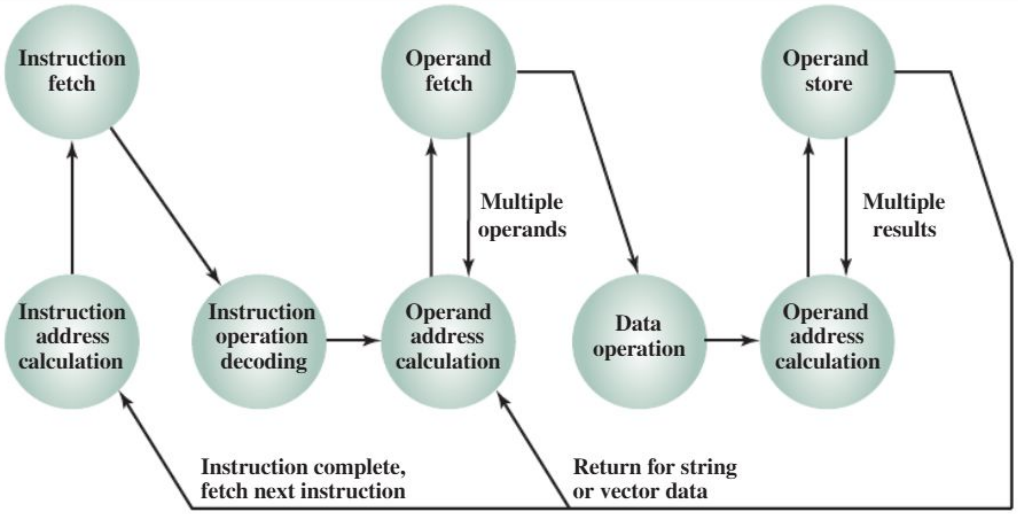


Figure 12.1 Instruction Cycle State Diagram

Source and result operands can be in one of four areas:

■ Main or virtual memory: As with next instruction references, the main or virtual memory address must be supplied.

■ Processor register: With rare exceptions, a processor contains one or more registers that may be referenced by machine instructions. If only one register exists, reference to it may be implicit. If more than one register exists, then each register is assigned a unique name or number, and the instruction must contain the number of the desired register.

■ Immediate: The value of the operand is contained in a field in the instruction being executed.

■ I/O device: The instruction must specify the I/O module and device for the operation. If memory-mapped I/O is used, this is just another main or virtual memory address.

Types of Operations in Machine Instructions

1. Data Transfer Instructions

Transfer instructions allow data to be transferred from one place to another without altering its content. A shared transfer may occur between the memory and processor registers or between the processor registers and input/output.

LOAD: Transfer of data from memory into the register.
IN: Receives data from an input device.
MOVE: Data transfer between registers.
OUT: Outputs data from the register.
PUSH: Pushes data from a register towards the top of the stack.
STORE: Transfer of data from the register to the memory.
POP: Fetches top data from register or stack memory.
XCHG: Transfers information between registers and memory.

2. Instructions for Data Manipulation

The Data manipulation elements of machine instruction

Arithmetic Instructions
Calculated by adding, subtracting, multiplying, dividing, incrementing and decrementing.
For instance, ADD, INC, MUL, DEC, SUB, DIV, etc.

Logical and Arithmetic Instructions
The function performs arithmetic + shift left and arithmetic + shift right.

Logical Instructions
Bit-wise operations like AND, NOT, Exclusive-OR, OR, shift, and rotate are performed.
Example: NOT, AND, ROL, XOR, OR, SHL, ROR SHR, etc.

3. Instructions for Program Control

A program control instruction specifies a condition that affects the program counter, while a data transfer and manipulation Type of machine instruction specifies a condition for data processing operations. A program counter's value can change over time.
A break in the instruction execution sequence occurs when a program control instruction is executed.
Examples:

BRANCH - BR

SKIP - SKP

CALL - CALL

TEST - TST

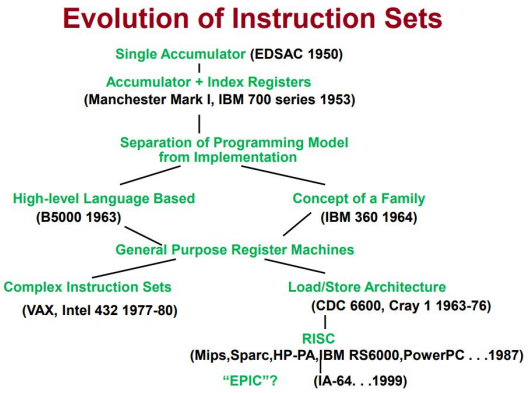
4. Program Interrupt

Interrupts are used to solve problems that occur outside of normal program sequences. When an external or internal element of machine instruction is generated, a program interrupt is used to transfer program control from one service program to another.

After the service program is executed, control returns to the original program. There are two types of interrupts: maskable interrupts and non-maskable interrupts.

Depending on the device, there can be external, internal, or software interrupts.

Brief History of the ISA



Accumulator (before 1960):

1 address	add A	acc <– acc + mem[A]

Stack (1960s to 1970s):

0 address	add	tos <– tos + next

Memory-Memory (1970s to 1980s):

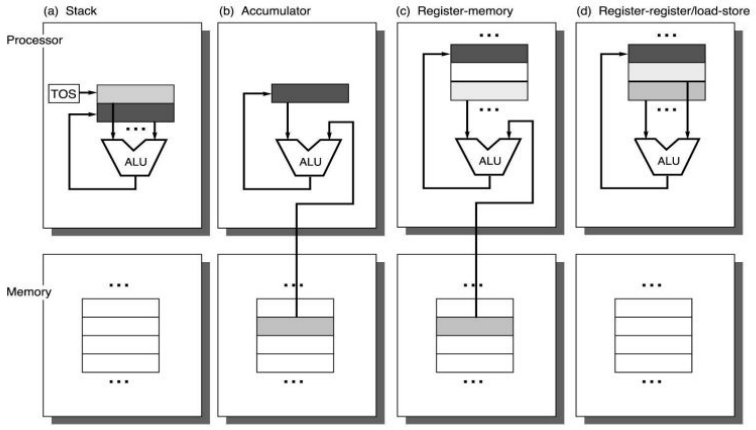
2 address	add A, B	mem[A] <– mem[A] + mem[B]
3 address	add A, B, C	mem[A] <– mem[B] + mem[C]

Register-Memory (1970s to present):

2 address	add R1, A	R1 <– R1 + mem[A]
	load R1, A	R1 <_ mem[A]

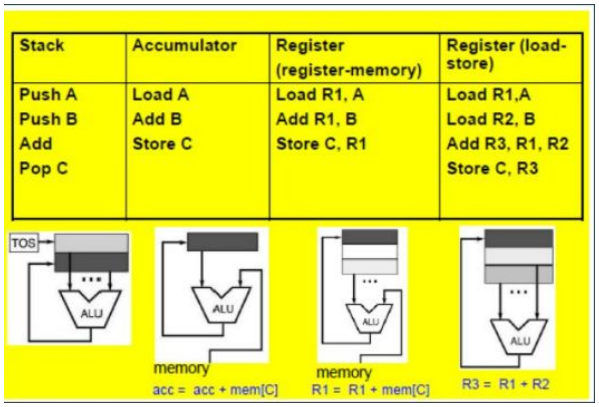
Register-Register (Load/Store) (1960s to present):

3 address	add R1, R2, R3	R1 <– R2 + R3
	load R1, R2	R1 <– mem[R2]
	store R1, R2	mem[R1] <– R2



Comparison of Functionality

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1, A	Load R1,A
Push B	Add B	Add R1, B	Load R2, B
Add	Store C	Store C, R1	Add R3, R1, R2
Pop C			Store C, R3



Instruction Set Design

The design of the instruction set is on of the most analysed aspects of computer design. A programmer's requirements must be considered when designing an instruction set, as it is the programmer's means of controlling the processor.

The design of the instruction set can be summed up as: what should be included in the instruction set (what is a must for the machine), and what can be left as an option, how do instructions look like and what is the relation between hardware and the instruction set

Fundamental design issues in an ISA

■ Operation repertoire: How many and which operations to provide, and how complex operations should be.

■ Data types: The various types of data upon which operations are performed.

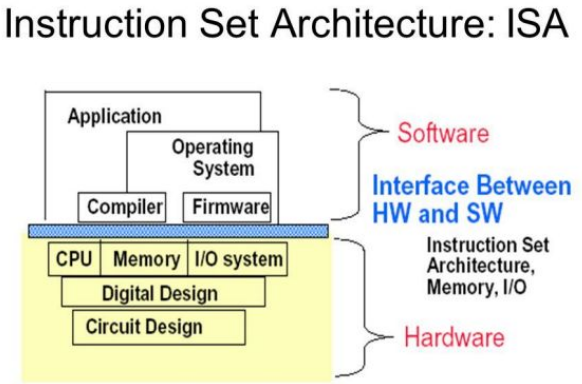
■ Instruction format: Instruction length (in bits), number of addresses, size of various fields, and so on.

■ Registers: Number of processor registers that can be referenced by instructions, and their use.

■ Addressing: The mode or modes by which the address of an operand is specified.

What is the Instruction Set Architecture?

First try to explain in your own words what an ISA is?



Definition of ISA

An instruction set architecture (ISA) is the abstract interface between a processor's hardware and the software it runs. It defines the supported instructions, data types, registers, memory access models, and I/O handling, enabling software compatibility across different implementations of a processor family.

An ISA defines the programmer-visible behavior of the processor. Software written in high-level languages is compiled into machine code that conforms to a specific ISA. The processor then executes this machine code by decoding and performing the defined operations. Despite differences in microarchitecture, processors that implement the same ISA can run the same binary software, enabling portability and hardware abstraction. ISAs can be extended to support new capabilities while maintaining backward compatibility. This is crucial for evolving processor designs without breaking existing software.

Key ISA Components

- Instruction set: The full list of machine instructions supported by the processor (e.g., arithmetic, control flow, memory operations)
- Instruction formats: Bit-level layout defining opcodes, operands, and addressing modes
- Registers: Number, types, and roles (e.g., general purpose, floating point, special function)
- Data types: Supported types like integers, floating-point numbers, vectors, or packed data
- Memory architecture: Includes addressing modes, endianness, memory protection, and virtual memory support
- Privilege levels: Execution modes such as user mode and kernel mode to enable secure OS functionality
- Interrupts and exceptions: Mechanisms for handling asynchronous events and fault conditions

2. Addressing Modes and Formats

An instruction contains an operation field, an address field, and a mode field. The operation field indicates what operation is to be performed, for example, addition, subtraction, multiplication, etc. The mode field indicates how the memory address of the operand, which is to be used in an operation, is determined. To understand the various types of addressing modes, it is important to understand how the computer deals with instruction.

First, virtually all computer architectures provide more than one of these addressing modes. The question arises as to how the processor can determine which address mode is being used in a particular instruction. Several approaches are taken. Often, different opcodes will use different addressing modes.

The second comment concerns the interpretation of the effective address (EA). In a system without virtual memory, the effective address will be either a main memory address or a register. In a virtual memory system, the effective address is a virtual address or a register. The actual mapping to a physical address is a function of the memory management unit (MMU) and is invisible to the programmer.

Common Addressing Techniques

1. Implicit
2. Immediate
3. Direct
4. Indirect
5. Register
6. Register Indirect
7. Displacement
    I. Relative
    II. Base-register
    III. Indexing
8. Stack

Table 13.1 Basic Addressing Modes

Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory reference	Limited address space
Register indirect	EA = (R)	Large address space	Extra memory reference
Displacement	EA = A + (R)	Flexibility	Complexity
Stack	EA = top of stack	No memory reference	Limited applicability

A = contents of an address field in the instruction
R = contents of an address field in the instruction that refers to a register
EA = actual (effective) address of the location containing the referenced operand
(X) = contents of memory location X or register X

1. Implicit Addressing Mode

In this addressing mode, the definition of the instruction itself specify the operands implicitly.

Examples:

The instruction “Complement Accumulator” is an implied mode instruction (CMA).
In a stack organized computer, Zero Address Instructions are implied mode instructions. (since operands are always implied to be present on the top of the stack)

2. Immediate Addressing Mode

In this addressing mode, the operand is specified in the instruction explicitly. Instead of address field, an operand field is present that contains the operand.

Examples:

ADD 10 will increment the value stored in the accumulator by 10.
MOV R #20 initializes register R to a constant value 20.

3. Direct Addressing Mode

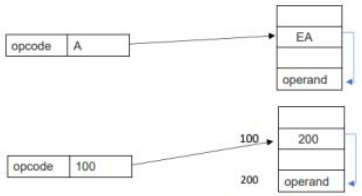
In this addressing mode, the address field of the instruction contains the effective address of the operand. Only one reference to memory is required to fetch the operand. It is also called as absolute addressing mode.
Example:

ADD X will increment the value stored in the accumulator by the value stored at memory location X.
AC ← AC + [X]

4. Indirect Addressing Mode

In this addressing mode, the address field of the instruction specifies the address of memory location that contains the effective address of the operand. Two references to memory are required to fetch the operand.

Example:
ADD X will increment the value stored in the accumulator by the value stored at memory location specified by X.
AC ← AC + [[X]]

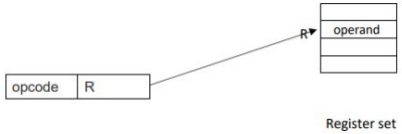


5. Register Addressing Mode
In this addressing mode, the operand is contained in a register set. The address field of the instruction refers to a CPU register that contains the operand. No reference to memory is required to fetch the operand.

Example:

ADD R will increment the value stored in the accumulator by the content of register R.
AC ← AC + [R]

This addressing mode is similar to direct addressing mode. The only difference is address field of the instruction refers to a CPU register instead of main memory.



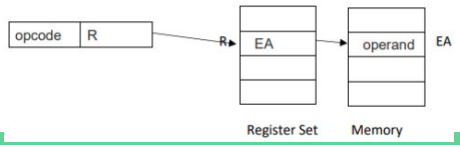
6. Register Indirect Addressing Mode

In this addressing mode, the address field of the instruction refers to a CPU register that contains the effective address of the operand. Only one reference to memory is required to fetch the operand.

Example:

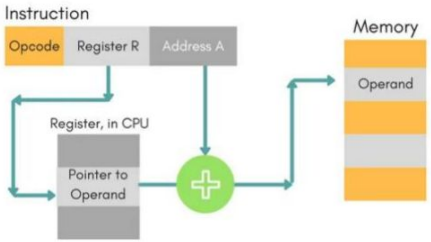
ADD R will increment the value stored in the accumulator by the content of memory location specified in register R.
AC ← AC + [[R]]

This addressing mode is similar to indirect addressing mode. The only difference is address field of the instruction refers to a CPU register.



7. Displacement Addressing Mode

In this the contents of the indexed register is added to the Address part of the instruction, to obtain the effective address of operand.
EA = A + (R), In this the address field holds two values, A(which is the base value) and R(that holds the displacement), or vice versa.



Relative Addressing (PC-Relative Addressing):

- Implicit Register: Program Counter (PC).
- Effective Address (EA): Calculated as EA = PC + Address Field.
- Purpose: Address is a displacement relative to the current instruction address.
- Use Case: Commonly used in branch instructions.

Base-Register Addressing:

- Base Register: Contains a main memory address.
- Displacement: Address field contains an offset (usually unsigned) from the base address.
- Effective Address (EA): Calculated as EA = Base Register + Address Field.
- Register Reference: Can be explicit or implicit.
- Use Case: Useful for accessing data structures like arrays or records.

Indexing:

- Address Field: References a main memory address.
- Index Register: Contains a positive displacement from the address field.
- Effective Address (EA): Calculated as EA = Address Field + Index Register.
- Key Difference: Opposite of base-register addressing (address field is the base, and index register is the offset).
- Use Case: Efficient for iterative operations (e.g., looping through arrays).

8. Stack Addressing Mode

In this addressing mode, the operand is contained at the top of the stack.

Example:
ADD

- This instruction simply pops out two symbols contained at the top of the stack.
- The addition of those two operands is performed.
- The result so obtained after addition is pushed again at the top of the stack.

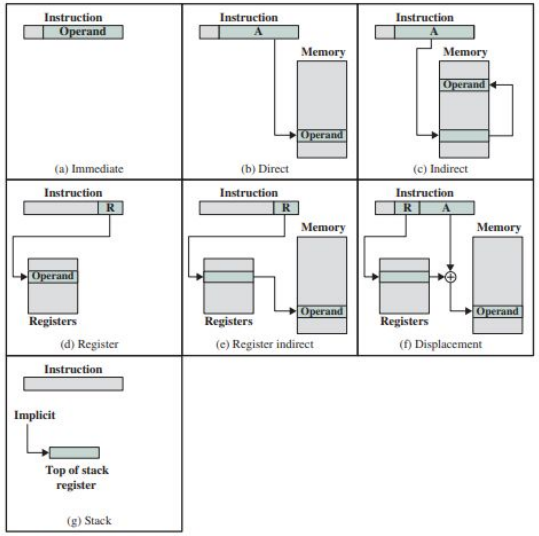


Figure 13.1 Addressing Modes

RISC vs. CISC, what is the difference?

What is RISC?

Reduced Instruction Set Computing (RISC) takes a streamlined approach, focusing on simplicity and speed. In RISC architectures like ARM and RISC-V, instructions are designed to be completed in a single clock cycle. This simplicity enables faster execution and lower power consumption, making RISC ideal for mobile devices, embedded systems, and efficient computing.

In ARM assembly, a series of instructions separates data loading and processing.

```
ldr r0, [r1]    ; Load value from memory at address in r1 to r0
add r0, r0, #1  ; Add 1 to r0
str r0, [r1]    ; Store the result back in memory at address in r1
```

Key Characteristics of RISC

1. Simple, Single-Cycle Instructions: RISC instructions are generally simple, allowing them to execute quickly and predictably.
2. Fixed-Length Instructions: Typically, RISC instructions are of uniform length, simplifying the decoding process and enhancing performance.
3. Load/Store Architecture: RISC separates memory access and computation, requiring data to be loaded into registers first, then operated on.
4. Limited Addressing Modes: RISC has fewer addressing modes, reducing complexity but potentially requiring additional instructions to perform complex operations.

What is CISC?

Complex Instruction Set Computing (CISC) is a processor architecture that prioritizes multi-step, powerful instructions. This approach was designed to minimize the number of instructions a programmer needs to write by offering instructions that can perform several tasks at once. CISC architectures like x86 (used by Intel and AMD) are common in personal computers and servers.

In x86 assembly, a single instruction may combine data loading and arithmetic

```
mov eax, [ebx]
```

Key Characteristics of CISC

1. Complex, Multi-Step Instructions: Many CISC instructions perform multiple operations in one step. This reduces the number of instructions required but often requires more time per instruction.
2. Variable-Length Instructions: Instructions can vary in length (some as short as 1 byte, others as long as several bytes), making decoding more complex but allowing flexibility.
3. Direct Memory Access: CISC instructions can access memory directly, which simplifies programming but requires additional time to access data.
4. Rich Addressing Modes: CISC supports various addressing methods for accessing data, providing flexibility for managing data structures.

3. MIPS Architecture

Microprocessor without Interlocked Pipeline Stages, MIPS is a type of microprocessor architecture that utilizes a reduced instruction set computing (RISC) approach. In simpler terms, it's a way of organizing how a computer's processor works, focusing on simplicity and speed.

MIPS processor implemented a smaller, simpler instruction set. Each of the instructions included in the chip design ran in a single clock cycle. The processor used a technique called pipelining to more efficiently process instructions.

MIPS used 32 registers, each 32 bits wide (a bit pattern of this size is referred to as a word). The MIPS instruction set consists of about 111 total instructions, each represented in 32 bits

In computing, MIPS works by executing simple instructions in a single clock cycle. It uses a pipeline to arrange the instructions so that every step of an instruction (fetch, decode, execute, etc.) is handled in a different stage of the pipeline. This allows for efficient and fast execution of commands.

It's often used in embedded systems like home routers, video game consoles, and digital TVs due to its efficiency and low power consumption. It's also used in some supercomputers because of its high performance.

MIPS Register Convention

Register Number	Conventional Name	Usage
$0	$zero	Hard-wired to 0
$1	$at	Reserved for pseudo-instructions
$2 - $3	$v0, $v1	Return values from functions
$4 - $7	$a0 - $a3	Arguments to functions - not preserved by subprograms
$8 - $15	$t0 - $t7	Temporary data, not preserved by subprograms
$16 - $23	$s0 - $s7	Saved registers, preserved by subprograms
$24 - $25	$t8 - $t9	More temporary registers, not preserved by subprograms
$26 - $27	$k0 - $k1	Reserved for kernel. Do not use.
$28	$gp	Global Area Pointer (base of global data segment)
$29	$sp	Stack Pointer
$30	$fp	Frame Pointer
$31	$ra	Return Address
$f0 - $f3	-	Floating point return values
$f4 - $f10	-	Temporary registers, not preserved by subprograms
$f12 - $f14	-	First two arguments to subprograms, not preserved by subprograms
$f16 - $f18	-	More temporary registers, not preserved by subprograms
$f20 - $f31	-	Saved registers, preserved by subprograms

MIPS Register File

MIPS has 32 general-purpose registers and another 32 floating-point registers. Registers all begin with a dollar-symbol ($). The floating point registers are named $f0, $f1, ..., $f31. The general-purpose registers have both names and numbers, and are listed below. When programming in MIPS assembly, it is usually best to use the register names.
The Register file holds,
Two read ports: Allows two registers to be read simultaneously.
One write port: Allows one register to be written at a time.

31 of these are general-purpose registers that can be used in any of the instructions. The last one, denoted register zero, is defined to contain the number zero at all times.
Even though any of the registers can theoretically be used for any purpose, MIPS programmers have agreed upon a set of guidelines that specify how each of the registers should be used. Programmers (and compilers) know that as long as they follow these guidelines, their code will work properly with other MIPS code.

- Zero Register - The zero register ($zero or $0) always contains a value of 0. It is built into the hardware and therefore cannot be modified.
- $at Register - The $at (Assembler Temporary) register is used for temporary values within pseudo commands. It is not preserved across function calls. For example, with the (slt $at, $a0, $s2) command, $at is set to one if $a0 is less than $s2, otherwise it is set to zero.
- $v Registers - The $v Registers are used for returning values from functions. They are not preserved across function calls.
- Argument Registers - The $a registers are used for passing arguments to functions. They are not preserved across function calls.
- Temporaries - The temporary registers are used by the assembler or assembly language programmer to store intermediate values. They are not preserved across function calls.
- Saved Temporaries - Saved Temporary registers are used to store longer lasting values. They are preserved across function calls.
- $k Registers - The k registers are reserved for use by the OS kernel. They may change randomly at any time as they are used by interrupt handlers.
- Pointer Registers
    - Global Pointer ($gp) - Usually stores a pointer to the global data area (such that it can be accessed with memory offset addressing).
    - Stack Pointer ($sp) - Used to store the value of the stack pointer.
    - Frame Pointer ($fp) - Used to store the value of the frame pointer.
    - Return Address ($ra) - Stores the return address (the location in the program that a function needs to return to).
    - All Pointer Registers are preserved across function calls.

Thank You

12/05/2026